

BEE 4750 Homework 1: Introduction to Using Julia

Name: Sarah Vafiadis, Ella Hartmanis, Nathan Rhoads

ID: sv439, ,

Due Date

Thursday, 9/11/25, 9:00pm

Overview

Instructions

- Problem 1 consist of a series of code snippets for you to interpret and debug. You will be asked to identify relevant error(s) and fix the code.
- Problem 2 asks you to write code to implement several simple functions using more general programming and some which are unique to Julia (which might mean that you need to look at the documentation or use Julia's help function, which is accessed with ?, e.g. `?sum` for help with the `sum()` function) or to analyze Julia syntax.
- Problem 3 asks you to convert a verbal description of a wastewater treatment system into a Julia function, and then to use that function to explore the impact of different wastewater allocation strategies.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

Activating project at `~/Documents/GitHub/bee4750-hw1-sarah-and-nathan`

Installed EarCut_jll	v2.2.4+0
Installed PDMats	v0.11.35
Installed HypergeometricFunctions	v0.3.28
Installed StatsFuns	v1.5.0
Installed OffsetArrays	v1.17.0
Installed NetworkLayout	v0.4.10
Installed Unitful	v1.23.1
Installed StaticArrays	v1.9.14
Installed Distances	v0.10.12
Installed ArnoldiMethod	v0.4.0
Installed NLSolve	v4.5.1
Installed StaticArraysCore	v1.4.3
Installed ADTypes	v1.17.0
Installed FiniteDiff	v2.28.1
Installed SpecialFunctions	v2.5.1
Installed Ratios	v0.4.5
Installed NLSolversBase	v7.10.0
Installed CommonSubexpressions	v0.3.1
Installed Rmath_jll	v0.5.1+0
Installed ColorTypes	v0.11.5
Installed GraphRecipes	v0.5.14
Installed GeometryTypes	v0.8.5
Installed Graphs	v1.13.0
Installed DiffRules	v1.15.1
Installed DiffResults	v1.1.0
Installed Inflate	v0.1.5
Installed ColorVectorSpace	v0.10.0
Installed ConstructionBase	v1.6.0
Installed Extents	v0.1.6
Installed Setfield	v1.1.2
Installed AbstractTrees	v0.4.5
Installed UnPack	v1.0.2
Installed OpenSpecFun_jll	v0.5.6+0
Installed ArrayInterface	v7.20.0
Installed SimpleTraits	v0.9.4
Installed LineSearches	v7.4.0
Installed DifferentiationInterface	v0.7.7
Installed Adapt	v4.3.0
Installed ChainRulesCore	v1.25.2
Installed Rmath	v0.8.0
Installed IterTools	v1.10.0
Installed GeometryBasics	v0.5.10

```

Installed ForwardDiff          v1.2.0
Installed Interpolations       v0.16.1
Installed AxisAlgorithms       v1.1.0
Installed QuadGK               v2.11.2
Installed WoodburyMatrices     v1.0.0
Installed Parameters           v0.12.3
Installed FillArrays           v1.13.0
Installed Distributions         v0.25.120
Precompiling project...
  437.6 ms  UnPack
  647.6 ms  ADTypes
  565.5 ms  IterTools
  903.8 ms  OffsetArrays
  933.4 ms  FillArrays
  452.6 ms  Extents
  507.7 ms  AbstractTrees
  735.1 ms  Distances
  343.2 ms  Ratios
  432.2 ms  Inflate
  383.9 ms  Adapt
  441.1 ms  ConstructionBase
  355.9 ms  StaticArraysCore
  472.0 ms  CommonSubexpressions
  661.9 ms  WoodburyMatrices
  901.5 ms  PDMats
  625.1 ms  SimpleTraits
  507.0 ms  Rmath_jll
  451.0 ms  EarCut_jll
 1083.0 ms  ChainRulesCore
  520.4 ms  OpenSpecFun_jll
 1866.4 ms  ColorTypes
  927.2 ms  QuadGK
  758.6 ms  Parameters
  832.0 ms  FillArrays → FillArraysStatisticsExt
  543.0 ms  FillArrays → FillArraysSparseArraysExt
  654.9 ms  Distances → DistancesSparseArraysExt
 1302.2 ms  DifferentiationInterface
  529.6 ms  Ratios → RatiosFixedPointNumbersExt
  682.0 ms  Adapt → AdaptSparseArraysExt
  489.6 ms  ArrayInterface
  548.6 ms  OffsetArrays → OffsetArraysAdaptExt
  575.0 ms  ConstructionBase → ConstructionBaseLinearAlgebraExt
  403.4 ms  ADTypes → ADTypesConstructionBaseExt

```

496.9 ms	DiffResults
713.4 ms	AxisAlgorithms
731.6 ms	FillArrays → FillArraysPDMatsExt
1102.9 ms	Rmath
703.6 ms	ChainRulesCore → ChainRulesCoreSparseArraysExt
406.3 ms	ADTypes → ADTypesChainRulesCoreExt
423.2 ms	Distances → DistancesChainRulesCoreExt
1074.1 ms	LogExpFunctions → LogExpFunctionsChainRulesCoreExt
1896.0 ms	ColorVectorSpace
2425.6 ms	SpecialFunctions
603.9 ms	DifferentiationInterface → DifferentiationInterfaceChainRulesCoreExt
585.1 ms	DifferentiationInterface → DifferentiationInterfaceSparseArraysExt
734.1 ms	ArrayInterface → ArrayInterfaceSparseArraysExt
394.3 ms	ArrayInterface → ArrayInterfaceChainRulesCoreExt
387.6 ms	ArrayInterface → ArrayInterfaceStaticArraysCoreExt
3741.9 ms	Colors
5987.2 ms	StaticArrays
953.4 ms	Setfield
419.9 ms	DiffRules
694.9 ms	ColorVectorSpace → SpecialFunctionsExt
1280.1 ms	HypergeometricFunctions
1791.1 ms	SpecialFunctions → SpecialFunctionsChainRulesCoreExt
690.6 ms	StaticArrays → StaticArraysChainRulesCoreExt
628.9 ms	StaticArrays → StaticArraysStatisticsExt
1148.6 ms	ArnoldiMethod
625.5 ms	Adapt → AdaptStaticArraysExt
665.2 ms	ConstructionBase → ConstructionBaseStaticArraysExt
661.8 ms	DifferentiationInterface → DifferentiationInterfaceStaticArraysExt
621.6 ms	FiniteDiff
3418.0 ms	ColorSchemes
1695.5 ms	StatsFuns
1653.1 ms	GeometryTypes
3188.5 ms	ForwardDiff
16092.8 ms	Unitful
634.1 ms	FiniteDiff → FiniteDiffSparseArraysExt
594.9 ms	FiniteDiff → FiniteDiffStaticArraysExt
498.6 ms	DifferentiationInterface → DifferentiationInterfaceFiniteDiffExt
2840.5 ms	Interpolations
3531.6 ms	Graphs
1304.6 ms	StatsFuns → StatsFunsChainRulesCoreExt
716.1 ms	DifferentiationInterface → DifferentiationInterfaceForwardDiffExt
925.8 ms	ForwardDiff → ForwardDiffStaticArraysExt
849.5 ms	Unitful → ConstructionBaseUnitfulExt

```

794.7 ms    Unitful → PrintfExt
653.5 ms    Unitful → ForwardDiffExt
1171.5 ms   Interpolations → InterpolationsForwardDiffExt
9544.9 ms   GeometryBasics
990.9 ms    Interpolations → InterpolationsUnitfulExt
1382.7 ms   NLSolversBase
1210.7 ms   UnitfulLatexify
4694.6 ms   Distributions
1533.3 ms   NetworkLayout
1101.9 ms   Distributions → DistributionsChainRulesCoreExt
1204.9 ms   Distributions → DistributionsTestExt
1566.3 ms   LineSearches
7626.7 ms   PlotUtils
1455.3 ms   NetworkLayout → NetworkLayoutGraphsExt
961.9 ms    NLSolve
1932.7 ms   PlotThemes
2441.3 ms   GraphRecipes
2646.9 ms   RecipesPipeline
40681.3 ms  Plots
3882.9 ms   Plots → UnitfulExt
4027.8 ms   Plots → GeometryBasicsExt
4433.7 ms   Plots → IJuliaExt
99 dependencies successfully precompiled in 74 seconds. 178 already precompiled.

```

Standard Julia practice is to load all needed packages at the top of a file. If you need to load any additional packages in any assignments beyond those which are loaded by default, feel free to add a `using` statement, though [you may need to install the package](#).

```

using Random
using Plots
using GraphRecipes
using LaTeXStrings
using Distributions

```

```

# this sets a random seed, which ensures reproducibility of random number generation. You should
Random.seed!(1)

```

```
TaskLocalRNG()
```

Problems (Total: 30 Points)

Problem 1 (12 points)

The following subproblems all involve code snippets that require debugging. You are encouraged to use online resources (*e.g.* Julia documentation and forums, Stack Overflow, Reddit, etc) to help find diagnose error messages and find solutions, but make sure that you clearly document which resources you used and how you used them.

Problem 1.1

You've been tasked with writing code to identify the minimum value in an array. You cannot use a predefined function. Your colleague suggested the function below, but it does not return the minimum value.

```
function minimum(array)
    # initialize the minimum value counter
    min_value = 0
    # update minimum values
    for i in 1:length(array)
        if array[i] < min_value
            min_value = array[i]
        end
    end
    # return found minimum
    return min_value
end

array_values = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
@show minimum(array_values);
```

```
minimum(array_values) = 0
```

What is the logical flaw in the code? Fix it and show that your fixed code solves the problem.

Problem 1.2

Your team is trying to compute the average grade for your class, but the following code produces an `UndefVarError`.

```
# enter student grade vector
student_grades = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
# compute class average
function class_average(grades)
    average_grade = mean(student_grades)
    return average_grade
end

@show average_grade;
```

```
UndefVarError: UndefVarError: `average_grade` not defined in `Main`
Suggestion: check for spelling errors or missing imports.
UndefVarError: `average_grade` not defined in `Main`
```

Suggestion: check for spelling errors or missing imports.

Stacktrace:

```
[1] macro expansion
```

```
    @ show.jl:1232 [inlined]
```

```
[2] top-level scope
```

```
@ ~/Documents/GitHub/bee4750-hw1-sarah-and-nathan/jl_notebook_cell_df34fa98e69747e1a8f8a7
```

What does this `UndefVarError` mean? Why does this code produce one? Fix the error and solve the problem.

Problem 1.3

Your team wants to know the expected payout of an old Italian dice game called *passadieci* (which was analyzed by Galileo as one of the first examples of a rigorous study of probability). The goal of *passadieci* is to get at least an 11 from rolling three fair, six-sided dice. Your strategy is to compute the average wins from 1,000 trials, but the code you've written below produces a `MethodError`.

```

# function to simulate a passadieci roll: 3 6-sided dice
function passadieci()
    # this rand() call samples 3 values from the vector [1, 6]
    roll = rand(1:6, 3)
    return roll
end
# set number of trials and initialize outcome vector
n_trials = 1_000
outcomes = zeros(n_trials)
# simulate number of passadieci rolls and count wins
for i = 1:n_trials
    outcomes[i] = (sum(passadieci()) > 11)
end
win_prob = sum(outcomes) / n_trials # compute average number of wins
@show win_prob;

```

```
win_prob = 0.393
```

The `MethodError` means that a function, in this case `win_prob`, was called and did not have all acceptable arguments. The `outcomes` vector was not a vector, it was a single value. I changed this by making `zero` plural, creating a vector of zeros with `n_trials` number of values. This made all the `win_prob` arguments work in the function and produced a valid probability.

What does this `MethodError` mean? Why does this code produce one? Fix the error and solve the problem.

Problem 1.4

You're interested in writing some code to remove the mean of a vector from all of its components. You've written the following code and tried to test it on a random vector, but your code returns a `MethodError`.

```

# function to remove mean from a vector
function remove_mean(vect)
    # fuction to compute the mean
    function compute_mean(vect)
        element_sum = 0 # initialize sum
        # compute mean and return
        for v in vect
            element_sum += v
        end
        return element_sum / length(vect)
    end
end

```



```

end

m = compute_mean(vect) # compute mean
# return demeaned vector
return vect .- m
end

random_vect = rand(1_000)
@show remove_mean(random_vect)

```

```
remove_mean(random_vect) = [0.05235532000260956, 0.03745694987919612, -0.12257223747856494, 0.29666150856190565, -0.15998996953340372, -0.021596389488312995, 0.21725179681475337, -0.3088185910217842, 0.4019264860719688, -0.005337752616891844, -0.252409435376873, 0.20632612169034048, 0.1618687539122574, -0.3889086271142378, -0.1813405323261691, -0.1223164636417825, -0.20184156887789173, -0.4291469925858571, -0.13082792844859514]
```

```
1000-element Vector{Float64}:
```

```

0.05235532000260956
0.03745694987919612
-0.12257223747856494
0.29666150856190565
-0.15998996953340372
-0.021596389488312995
0.21725179681475337
-0.3088185910217842
0.4019264860719688
-0.005337752616891844

-0.252409435376873
0.20632612169034048
0.1618687539122574
-0.3889086271142378
-0.1813405323261691
-0.1223164636417825
-0.20184156887789173
-0.4291469925858571
-0.13082792844859514

```

This `MethodError` means that there was a line that completed a math function that did not have the same arguments. In this example, a vector was subtracting a scalar, which cannot happen. In order to fix this, the scalar must have the same argument in order to subtract the mean from each individual component of the vector. According the stack overflow, this can be accomplished in Julia by adding a period in front of the subtraction symbol. The period lets the computer know that the operation should be applied to each component. This makes the rest of the function able to work

What does this `MethodError` mean? Why does this code produce one? Fix the error and solve the problem.

Problem 2 (18 points)

Cheap Plastic Products, Inc. is operating a plant that produces $100\text{m}^3/\text{day}$ of wastewater that is discharged into Pristine Brook. The wastewater contains $1\text{kg}/\text{m}^3$ of YUK, a toxic substance. The US Environmental Protection Agency has imposed an effluent standard on the plant prohibiting discharge of more than $20\text{kg}/\text{day}$ of YUK into Pristine Brook.

Cheap Plastic Products has analyzed two methods for reducing its discharges of YUK. Method 1 is land disposal, which costs $X_1^2/20$ dollars per day, where X_1 is the amount of wastewater disposed of on the land (m^3/day). With this method, 20% of the YUK applied to the land will eventually drain into the stream (*i.e.*, 80% of the YUK is removed by the soil).

Method 2 is a chemical treatment procedure which costs $\$1.50$ per m^3 of wastewater treated. The chemical treatment has an efficiency of $e = 1 - 0.005X_2$, where X_2 is the quantity of wastewater (m^3/day) treated. For example, if $X_2 = 50\text{m}^3/\text{day}$, then $e = 1 - 0.005(50) = 0.75$, so that 75% of the YUK is removed.

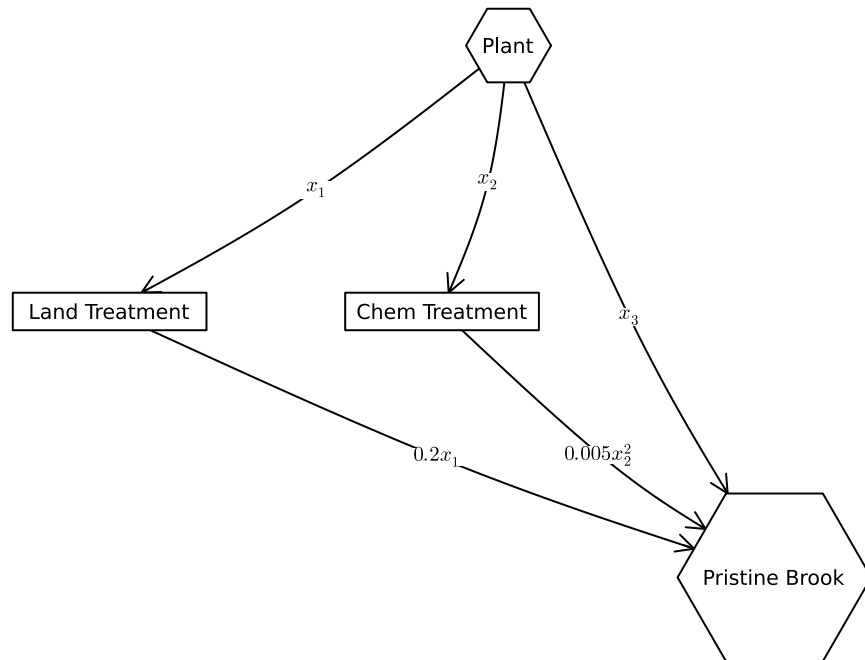
Cheap Plastic Products is wondering how to allocate their wastewater between these three disposal and treatment methods (land disposal, chemical treatment, and direct disposal) to meet the effluent standard while keeping costs manageable.

The flow of wastewater through this treatment system is shown in Figure 1. Modify the edge labels (by editing the `edge_labels` dictionary in the code producing Figure 1) to show how the wastewater allocations result in the final YUK discharge into Pristine Brook. For the `edge_label` dictionary, the tuple (i, j) corresponds to the arrow going from node i to node j . The syntax for any entry is $(i, j) \Rightarrow \text{"label text"}$, and the label text can include mathematical notation if the string is prefaced with an L, as in `L"x_1"` will produce x_1 .

```
A = [0 1 1 1;
      0 0 0 1;
      0 0 0 1;
      0 0 0 0]

names = ["Plant", "Land Treatment", "Chem Treatment", "Pristine Brook"]
# modify this dictionary to add labels
edge_labels = Dict{(1, 2) => L"x_1", (1,3) => L"x_2", (1, 4) => L"x_3", (2, 4) => L"0.2x_1", (3, 4) => L"0.75x_2"}
shapes=[:hexagon, :rect, :rect, :hexagon]
xpos = [0, -1.5, -0.25, 1]
ypos = [1, 0, 0, -1]
```

```
p = graphplot(A, names=names, edgelabel=edge_labels, markersize=0.15, markershapes=shapes, ma
display(p)
```



Problem 2.1

Formulate a mathematical model for the treatment cost and the amount of YUK that will be discharged into Pristine Brook based on the wastewater allocations. This is best done with some equations and supporting text explaining the derivation. Make sure you include, as additional equations in the model, any needed constraints on relevant values. You can find some basics on writing mathematical equations using the LaTeX typesetting syntax [here](#), and a cheatsheet with LaTeX commands can be found on the course website's [Resources page](#).

Our solution

Variables and constraints

Volume of wastewater disposed on land: $x_1 \geq 0$

Volume of wastewater chemically treated: $x_2 \geq 0$

Volume of wastewater disposed directly from plant: $x_3 \geq 0$

Equations

Overall wastewater flow: $x_1 + x_2 + x_3 = 100$

The total volume of wastewater analyzed in this problem is 100 m^3 , so all three wastewater flows should be equal to that value.

YUK discharged into Pristine Brook: $0.2x_1 + 0.005x_2^2 + x_3 \geq 20$

When land treatment is used, 20% of the volume treated is drained into Pristine Brook. Therefore, the volume treated (x_1), should be multiplied by 0.2. Chemical treatment has an efficiency of $e = 1 - 0.005x_2$. This gives the percentage of wastewater diverged from Pristine Brook. To calculate the amount entering the brook, the equation $1 - e$ should be used. When the equation for e is substituted into this equation, the result is $0.005x_2^2$. x_3 is the amount of wastewater released directly from the plant into the brook. All of these values are multiplied by 1 kg/m^3 , the amount of YUK in each m^3 of water. Since this value is 1, the overall equation doesn't change. Finally, no more than 20 kg of YUK may be directed into the brook each day, so this equation must be less than or equal to 20.

Cost:

$$\$ x_1^2/2 + 1.5x_2 = \text{C\$}$$

This equation accounts for the costs of land and chemical treatment as given in the problem statement.

Problem 2.2

Implement your systems model as a Julia function which computes the resulting YUK discharge and cost for a particular treatment plan. You can return multiple values from a function with a [tuple](#), as in:

```
function multiple_return_values(x, y)
    return (x+y, x*y)
end

a, b = multiple_return_values(2, 5)
@show a;
@show b;
```

```
a = 7
b = 10
```

To evaluate the function over vectors of inputs, you can *broadcast* the function by adding a decimal `.` before the function arguments and accessing the resulting values by writing a *comprehension* to loop over the individual outputs in the vector:

```
x = [1, 2, 3, 4, 5]
y = [6, 7, 8, 9, 10]

output = multiple_return_values.(x, y)
a = [out[1] for out in output]
b = [out[2] for out in output]
@show a;
@show b;
```

```
a = [7, 9, 11, 13, 15]
b = [6, 14, 24, 36, 50]
```

Our solution

```
function calculate_flows(x_1,x_2)

    # x_3 is dependent on x_1 and x_2 inputs in order to minimize cost
    x_3 = 100 - x_1 - x_2

    # These equations are pulled directly from the system designed in 2.1
    yuk_dis = 0.2*x_1 + 0.005*x_2^2 + x_3 - 20
    cost = (x_1^2)/2 + 1.5*x_2

    # Check that YUK discharge into Pristine Brook is less than 20 kg/day; @assert command for
    @assert yuk_dis <= 20

    # Check that all flows are non-negative
    @assert x_1 >= 0
    @assert x_2 >= 0
    @assert x_3 >= 0

    return (yuk_dis, cost, x_1, x_2, x_3)
end
```

```

# Unpacks variables to be displayed; values input in calculate_flows may be changed to find c
(yuk_dis, cost, x_1, x_2, x_3) = calculate_flows(40,40)

# Displays solution variables
@show yuk_dis
@show cost
@show x_1
@show x_2
@show x_3

```

```

InexactError: InexactError: Int64(NaN)
InexactError: Int64(NaN)

```

Stacktrace:

```

[1] Int64(x::Float64)

    @ Base .\float.jl:994

[2] alloc_DF(x::Vector{Int64}, F::Vector{Int64})

    @ NLSolversBase C:\Users\sarah\.julia\packages\NLSolversBase\n7XX0\src\objective_types\ab

[3] nlsolve(f::Function, initial_x::Vector{Int64}; method::Symbol, autodiff::Symbol, inplace

    @ NLSolve C:\Users\sarah\.julia\packages\NLSolve\gJL1I\src\nlsolve\nlsolve.jl:49

[4] nlsolve(f::Function, initial_x::Vector{Int64})

    @ NLSolve C:\Users\sarah\.julia\packages\NLSolve\gJL1I\src\nlsolve\nlsolve.jl:40

[5] top-level scope

    @ c:\Users\sarah\OneDrive\Documents\GitHub\bee4750-hw1-sarah-and-nathan\jl_notebook_cell_0

```

Make sure you comment your code appropriately to make it clear what is going on and why.

Problem 2.3

Use your function to experiment with 1,000 different combinations of wastewater discharge and treatment. You can do this with either a grid search or by sampling from a [Dirichlet distribution](#) (a $\text{Dirichlet}(1, n)$ distribution will generate uniformly-weighted n -dimensional vectors whose components add up to 1; see the [Distributions.jl documentation](#) for how to sample from probability distributions in Julia). Plot the results of these experiments. Do any satisfy the YUK effluent standard (plot this as well as a dashed red line). What was the cost of solutions satisfying the standard? What can you say about the tradeoff between treatment cost and YUK discharge? You don't have to find an "optimal" solution to this problem, but what do you think would be needed to find a better solution?

Problem 2.4

Find the strategies which minimize cost and YUK discharge (these will be different strategies) analytically and find the values of the objective metrics. Plot these values in the plot that you created for Problem 2.3. How do their values compare to the spread of values that you found in that problem? Would you select either of them (explain why or why not)?

References

List any external references consulted, including classmates.

Our Solution

```
using Plots
```